

Session 15 : Joints, Moteurs & Raycasts

Objectifs de la session

- Comprendre ce qu'est un **joint** (articulation) et comment il **contraint** le mouvement relatif de deux rigid bodies.
- Maîtriser les **quatre types fondamentaux** : fixed, revolute, prismatic, spherical — et choisir le bon selon le cas.
- Configurer des **moteurs** sur les joints pour les animer activement (vitesse ou position cible).
- Ajouter des **limites** (min/max) pour borner la course d'une articulation.
- Découvrir les **raycasts** comme outil de requête du moteur physique — détection du sol, visée, capteurs.
- Construire pas à pas une **catapulte fonctionnelle** dans Rapier + Three.js, puis la transposer en Godot.

💡 Le passage à l'articulation

La Session 14 a posé les **corps** — fixed, dynamic, kinematic — et les **colliders**. Mais jusqu'ici, chaque corps était **isolé** : un cube tombe, rebondit, s'empile. Aucun n'était **relié** à un autre. Les joints changent tout : ils permettent de **souder**, **charnière**, **glisser** et **pivoter** des corps entre eux. C'est ce qui transforme une boîte qui tombe en un bras de catapulte, un ragdoll, une porte, un véhicule.

Partie 1 : Qu'est-ce qu'un joint ?

Joint (articulation)

Un **joint** est une contrainte qui relie **deux** rigid bodies et **limite leurs degrés de liberté** relatifs. Sans joint, deux bodies ont 6 degrés de liberté relatifs (3 translations + 3 rotations). Un joint en **supprime** certains :

- **Fixed** : supprime les 6 — les deux corps sont soudés.
- **Revolute** : supprime 5 — il ne reste qu'une **rotation** autour d'un axe.
- **Prismatic** : supprime 5 — il ne reste qu'une **translation** le long d'un axe.
- **Spherical** : supprime 3 — il reste les 3 **rotations** (rotule).

Le moteur physique **résout** ces contraintes à chaque `step()`, au même titre que les contacts. Un joint est donc une **équation** ajoutée au solveur — et le solveur trouve les vitesses qui satisfont à **la fois** les contacts et les joints.

💡 Joints vs contraintes maison

En Session 7, on a codé un ressort à la main : une force $F = -k \cdot x$ qui rapproche deux points. Un joint est **l'équivalent rigide** : au lieu d'une force **proportionnelle** à l'écart, le solveur impose **zéro écart** — c'est une contrainte **dure**. Le ressort est **mou** (il oscille), le joint est **dur** (il ne cède pas). En pratique, on utilise des joints pour les articulations mécaniques et des ressorts pour les suspensions, tissus, objets souples.

Partie 2 : Les quatre types de joints

Fixed — la soudure

Fixed Joint

Le joint le plus simple : les deux corps sont **rigidement liés**. La distance et l'orientation relatives sont **figées**. C'est l'équivalent de souder deux pièces.

```
// Rapier - JointData.fixed(anchor1, frame1, anchor2, frame2)
const joint = world.createImpulseJoint(
  RAPIER.JointData.fixed(
    { x: 0, y: 0.5, z: 0 }, // ancre sur A (local)
    { x: 0, y: 0, z: 0, w: 1 }, // frame A (quaternion identité)
    { x: 0, y: -0.5, z: 0 }, // ancre sur B (local)
    { x: 0, y: 0, z: 0, w: 1 } // frame B
  ),
  bodyA, bodyB, true // true = réveille les corps
);
```

Les **frames** (quaternions) définissent l'orientation relative **souhaitée** entre les deux ancrs. Le identity signifie « alignées ». On peut les tourner pour souder deux corps avec un angle précis.

💡 Fixed joint vs un seul body multi-colliders

Pourquoi souder deux bodies plutôt que d'en faire un seul avec plusieurs colliders ? Parce qu'un joint permet de **casser** la liaison à l'exécution (`world.removeJoint(joint)`) — un pont qui s'effondre, un véhicule qui perd une roue. Un body multi-colliders est **insécable**. Le fixed joint est l'outil de la **destruction** et des **assemblages dynamiques**.

Revolute — la charnière

Revolute Joint

Le joint le plus utile en mécanique : les deux corps ne peuvent que **tourner** l'un par rapport à l'autre autour d'un **axe** défini. C'est la charnière d'une porte, l'axe d'une roue, le coude d'un bras.

```
// Rapier - JointData.revolute(anchor1, anchor2, axis)
const joint = world.createImpulseJoint(
  RAPIER.JointData.revolute(
    { x: 0, y: -1, z: 0 }, // ancre sur A (pied du bras)
    { x: 0, y: 1, z: 0 }, // ancre sur B (sommet de la base)
    { x: 0, y: 0, z: 1 } // axe de rotation (Z = avant/arrière)
  ),
  bodyA, bodyB, true
);
```

L'**axe** est exprimé dans le **repère local de A**. Le moteur aligne l'ancre de B sur l'ancre de A et autorise uniquement la rotation autour de cet axe. En 3D, l'axe compte : { x: 0, y: 1, z: 0 } = charnière verticale (porte), { x: 0, y: 0, z: 1 } = charnière horizontale (bras de catapulte).

Cas d'usage du revolute.

- **Porte** : un body fixe (mur) + un body dynamique (porte) + revolute axe Y.
- **Roue** : un body châssis + un body roue + revolute axe X.
- **Catapulte** : un body base + un body bras + revolute axe Z + moteur (Partie 3).

- **Ragdoll** : un body torse + un body bras + revolute axe Z + limites (Partie 4).

Prismatic — le tiroir

Prismatic Joint

Le pendant du revolute pour la translation : les deux corps ne peuvent que **glisser** l'un par rapport à l'autre le long d'un **axe**. C'est le tiroir d'une commode, le piston d'un vérin, la platine d'une presse.

```
// Rapier — JointData.prismatic(anchor1, anchor2, axis)
const joint = world.createImpulseJoint(
  RAPIER.JointData.prismatic(
    { x: 0, y: 0, z: 0 }, // ancre sur A
    { x: 0, y: 0, z: 0 }, // ancre sur B
    { x: 0, y: 1, z: 0 }  // axe de translation (Y = haut/bas)
  ),
  bodyA, bodyB, true
);
```

Le prismatic **bloque toute rotation** — contrairement au revolute qui bloque toute translation. C'est la contrainte **complémentaire** : l'un tourne, l'autre glisse.

Spherical — la rotule

Spherical Joint

Le joint le plus libre : les deux ancres coïncident, mais les corps peuvent **tourner librement** dans les trois axes. C'est l'épaule, la hanche, la rotule d'une suspension.

```
// Rapier — JointData.spherical(anchor1, anchor2)
const joint = world.createImpulseJoint(
  RAPIER.JointData.spherical(
    { x: 0, y: 0, z: 0 }, // ancre sur A
    { x: 0, y: 0, z: 0 }  // ancre sur B — pas d'axe !
  ),
  bodyA, bodyB, true
);
```

Pas d'axe — c'est la différence clé avec le revolute. Le spherical **ne contraint que la position** des ancres ; les rotations sont libres. On l'utilise pour les **chaînes** (maillons), les **ragdolls** (épaules), les **pendules** (balancier libre).

⚠ Spherical vs Revolute — le piège

Un débutant qui veut faire une porte utilise parfois un spherical joint « parce que ça tourne ». Mais le spherical autorise **toutes** les rotations — la porte se mettrait à pivoter sur tous ses axes comme un drapeau. Le revolute **restreint** à un seul axe. La règle : si le mouvement doit être **guidé** (un seul axe), c'est revolute. S'il doit être **libre** (rotule), c'est spherical.

Partie 3 : Les moteurs — animer un joint

Motor (moteur d'articulation)

Un joint **contraint** — un **moteur pousse**. On ajoute un moteur à un joint revolute ou prismatic pour qu'il **cherche activement** une cible : une **vitesse** (tourner à 5 rad/s) ou une **position** (aller à 30°). Le moteur applique un **couple** (torque) ou une **force** le long de l'axe libre du joint.

```
// Rapiere — moteur en VITESSE (tourne à 5 rad/s)
joint.configureMotorVelocity(
    5.0,      // targetVel : vitesse cible (rad/s pour revolute, m/s pour prismatic)
    1.0      // factor : "force" du moteur (0 = mou, ∞ = rigide)
);

// Rapiere — moteur en POSITION (va à  $\pi/4$  rad)
joint.configureMotorPosition(
    Math.PI / 4, // targetPos : angle cible
    1000.0,      // stiffness : raideur ( $\approx k$  du ressort de la S7)
    100.0        // damping : amortissement ( $\approx c$  du ressort)
);
```

Le moteur en **vitesse** est un **régulateur** : il maintient la vitesse quoi qu'il arrive (une roue qui tourne). Le moteur en **position** est un **servo** : il vise un angle précis (un bras robotique). Les deux paramètres stiffness et damping sont **exactement** le ressort de la Session 7 — k et c — mais résolu comme une **contrainte** au lieu d'une force.

💡 Le lien avec la Session 7

En Session 7, le ressort était $F = -k \cdot x - c \cdot v$. Le moteur de position d'un joint est **la même équation**, mais intégrée dans le solveur de contraintes au lieu d'être une force externe.

Conséquence : il est **plus stable** (pas d'oscillation explosive si k est grand) et **plus précis** (le solveur itère). C'est pourquoi on ne code plus les ressorts à la main quand on a un moteur physique — on utilise les moteurs de joints.

configureMotor — le mode combiné

Pour un contrôle fin, on peut combiner **position** et **vitesse** cibles :

```
joint.configureMotor(
    Math.PI / 4, // targetPos : angle cible
    0.0,        // targetVel : vitesse cible *en atteignant* la position
    1000.0,     // stiffness
    100.0       // damping
);
```

Le moteur vise l'angle $\pi/4$ **en se déplaçant** à la vitesse 0 (il ralentit en arrivant). C'est le mode **asservi** — l'équivalent d'un PID. On l'utilise pour les bras robotiques, les portes automatiques, les mécanismes précis.

Partie 4 : Les limites — borner la course

Limits (butées)

Un joint revolute ou prismatic peut être **borné** : on définit un angle (ou une translation) **minimum** et **maximum** au-delà desquels le corps ne peut pas aller. Le moteur ajoute une **butée** — l'équivalent du butoir mécanique.

```
// Rapiér — limites sur un revolute (bras de catapulte)
joint.setLimits(
  -Math.PI / 2, // min : -90° (bras vers le bas)
  Math.PI / 3   // max : +60° (bras vers le haut)
);
```

Les limites sont **activées par défaut** après setLimits. On peut les désactiver avec joint.setLimits(0, -1) (convention Rapiér : min > max = désactivé). En pratique, on les laisse activées — un joint sans limites peut tourner indéfiniment, ce qui est rarement souhaité.

! Limites + moteur = mécanisme complet

Une catapulte, c'est : un **revolute joint** + un **moteur en position** (qui tire le bras vers l'arrière) + des **limites** (qui empêchent le bras de dépasser la verticale). Quand le projectile quitte le bras, le moteur **relâche** (vitesse cible = 0) et le bras s'arrête aux limites. C'est exactement le TP de cette session.

Partie 5 : Les raycasts — interroger le monde

Raycast

Un **raycast** projette un **rayon** (un segment) dans le monde physique et retourne le **premier collider touché**, avec la **distance** et la **normale** du point d'impact. C'est l'outil de **requête** le plus utilisé — il ne **modifie** pas la simulation, il **lit** l'état du monde.

```
// Rapiér — raycast vertical depuis y=10 vers le bas
const ray = new RAPIER.Ray(
  { x: 0, y: 10, z: 0 }, // origine
  { x: 0, y: -1, z: 0 }  // direction (NORMALISÉE !)
);
const hit = world.castRay(ray, 20.0, true);
//                               maxDist ↑      ↑ solid (true = premier hit)
if (hit) {
  const point = ray.pointAt(hit.timeOfImpact); // { x, y, z }
  const normal = hit.normal;                   // { x, y, z }
  console.log("Touché à", hit.timeOfImpact, "m");
}
```

Le timeOfImpact est la **distance** (pas le temps — le rayon est normalisé). true = solid : on s'arrête au **premier** collider. false = on retourne **tous** les colliders touchés (via castRayAndGetNormal).

Cas d'usage des raycasts.

- **Détection du sol** : raycast vers le bas depuis le personnage → est-il au sol ? À quelle distance ?
- **Visée FPS** : raycast depuis la caméra dans la direction du regard → quel ennemi est touché ?
- **Capteur de véhicule** : raycast vers l'avant → obstacle à quelle distance ?

- **Placement d'objet** : raycast depuis le curseur → où poser un objet sur le sol ?
- **Suspension de roue** : raycast vers le bas depuis la roue → compression de l'amortisseur.

💡 Raycast physique vs raycast rendu

En Session 14, on a utilisé `THREE.Raycaster` pour convertir un clic souris en direction de tir. C'est un raycast **rendu** — il intersecte les **meshes** (visuels). Le raycast **physique** intersecte les **colliders** (simulés). Les deux sont utiles : le rendu pour « sur quoi clique le joueur », le physique pour « où est le sol sous le personnage ». En pratique, on utilise presque toujours le **physique** — les colliders sont plus simples et plus stables que les meshes.

Partie 6 : Exploration — Le Joint Zoo (Rapier + Three.js)

💡 L'exemple du cours

exemples/session15_rapier.html + session15_rapier.js — quatre stations indépendantes, côte à côte, une par type de joint. Chaque station est **isolée** (pas de collision entre corps articulés via `setContactsEnabled(false)`) pour garantir la stabilité. La GUI contrôle les moteurs et limites de chaque station en temps réel.

⚠️ Le piège universel : contacts entre corps articulés

Quand deux corps reliés par un joint se touchent **au niveau de l'axe d'articulation**, le solveur lutte entre la contrainte du joint et la réponse de collision — d'où un mouvement **erratique**. La solution : `joint.setContactsEnabled(false)` désactive les collisions entre les deux bodies du joint. C'est la **règle d'or** des articulations : on ne fait **jamais** collisionner deux corps reliés par un joint au niveau de leur point d'attache.

Station 1 — Le berce de Newton (Revolute, 5 balles)

Cinq pendules côte à côte, sans moteur

```
// --- Support fixe (barre horizontale) ---
const anchorBody = world.createRigidBody(
  RAPIER.RigidBodyDesc.fixed().setTranslation(-8, 5, 0)
);

// --- 5 balles, chacune suspendue par un revolute joint ---
for (let i = 0; i < 5; i++) {
  const x = -8 + (i - 2) * 0.72; // espacées de ~2x le rayon

  const ballBody = world.createRigidBody(
    RAPIER.RigidBodyDesc.dynamic()
      .setTranslation(x, 2.5, 0) // 2.5 = 5 - 2.5 (longueur du fil)
  );
  world.createCollider(
    RAPIER.ColliderDesc.ball(0.35)
      .setDensity(3.0) // acier — lourd
      .setRestitution(0.95), // quasi-élastique
    ballBody
  );
}
```

```
// --- JOINT REVOLUTE : balle suspendue au support ---
const joint = world.createImpulseJoint(
  RAPIER.JointData.revolute(
    { x: (i - 2) * 0.72, y: 0, z: 0 }, // ancre sur le support
    { x: 0, y: 2.5, z: 0 },           // ancre sur la balle (haut)
    { x: 0, y: 0, z: 1 }               // axe Z = pendule
  ),
  anchorBody, ballBody, true
);
joint.setContactsEnabled(false); // ← RÈGLE D'OR
}
```

Le berce de Newton est la démonstration la plus **élégante** du revolute : cinq pendules identiques, **sans moteur**, qui se balancent librement. Le bouton « Tirer » téléporte la première balle à un angle donné et la relâche — l'énergie traverse la rangée et la dernière balle s'envole. Les balles ont une **restitution** élevée (0.95) pour que les collisions soient quasi-élastiques. Pas de limites ici : les balles peuvent penduler librement.

💡 Pourquoi pas de moteur ?

Le berce de Newton est un système **passif** — l'énergie se conserve à travers les collisions. Ajouter un moteur **injecterait** de l'énergie et détruirait la démo. C'est l'inverse du pendule motorisé : ici, le revolute joint sert uniquement à **contraindre** le mouvement (un seul axe), pas à l'entraîner. Le moteur est un **composant optionnel** du joint, pas une obligation.

Station 2 — Le piston (Prismatic + moteur + limites)

Un piston glisse sur une pente et pousse une boîte

```
// --- Une seule pente inclinée à 20° (FIXE) ---
const slopeAngle = 20 * Math.PI / 180;
const slopeQuat = {
  x: 0, y: 0,
  z: Math.sin(slopeAngle / 2),
  w: Math.cos(slopeAngle / 2)
};
const slopeBody = world.createRigidBody(
  RAPIER.RigidBodyDesc.fixed()
  .setTranslation(-2, 2, 0)
  .setRotation(slopeQuat) // rotation Z = inclinaison
);
world.createCollider(
  RAPIER.ColliderDesc.cuboid(3, 0.15, 0.5)
  .setFriction(0.1), // glissant — la boîte redescend
  slopeBody
);

// --- Le piston (DYNAMIQUE, glisse le long de la pente) ---
const pistonBody = world.createRigidBody(
  RAPIER.RigidBodyDesc.dynamic()
  .setTranslation(pistonX, pistonY, 0)
  .setRotation(slopeQuat) // aligné avec la pente
  .setAngularDamping(5.0)
);
```

```

);

// --- LE JOINT PRISMATIC le long de la pente ---
// L'axe X local du slopeBody = direction de la pente (inclinée à 20°)
const pistonJoint = world.createImpulseJoint(
  RAPIER.JointData.prismatic(
    { x: -2, y: 0.5, z: 0 }, // ancre sur la pente (local)
    { x: 0, y: 0, z: 0 },    // ancre sur le piston (local)
    { x: 1, y: 0, z: 0 }    // axe X local = direction de la pente
  ),
  slopeBody, pistonBody, true
);
pistonJoint.setContactsEnabled(false);

// --- LIMITES + MOTEUR (oscille : pousse puis retracte) ---
pistonJoint.setLimits(-1.5, 2.0);
pistonJoint.configureMotorVelocity(2.0, 200);

// --- La boîte à pousser (corps LIBRE, pas de joint) ---
const boxBody = world.createRigidBody(
  RAPIER.RigidBodyDesc.dynamic()
    .setTranslation(boxX, boxY, 0)
    .setRotation(slopeQuat)
);
world.createCollider(
  RAPIER.ColliderDesc.cuboid(0.35, 0.35, 0.35)
    .setFriction(0.1), // glissant comme la pente
  boxBody
);

```

Le piston est le **complément** du berce : au lieu de **tourner**, il **glisse**. Ici, le prismatic est aligné avec la **pente** — l'axe X **local** du slopeBody, incliné à 20°, devient la direction de glissement dans le monde. Le piston et la boîte sont tous deux **rotés** par le même quaternion que la pente, pour rester alignés avec sa surface. Le moteur oscille (pousse 2 s, retracte 2 s) → la boîte monte la pente, puis redescend par gravité. La friction est volontairement **faible** (0.1) pour que la boîte glisse facilement.

Station 3 — La chaîne (Spherical, 4 maillons)

Une chaîne de sphères reliées par des rotules

```

// --- Anneau d'attache (FIXE, en haut) ---
const anchorBody = world.createRigidBody(
  RAPIER.RigidBodyDesc.fixed().setTranslation(4, 6, 0)
);

let prevBody = anchorBody;
let prevAnchor = { x: 0, y: -0.3, z: 0 }; // bas de l'anneau

for (let i = 0; i < 4; i++) {
  const y = 6 - (i + 1) * 0.6;

  // --- Maillon (DYNAMIQUE) ---
  const linkBody = world.createRigidBody(
    RAPIER.RigidBodyDesc.dynamic()

```

```

        .setTranslation(4, y, 0)
    );
    world.createCollider(
        RAPIER.ColliderDesc.ball(0.25), linkBody
    );

    // --- JOINT SPHERICAL entre ce maillon et le précédent ---
    const joint = world.createImpulseJoint(
        RAPIER.JointData.spherical(
            prevAnchor, // ancre sur le précédent
            { x: 0, y: 0.25, z: 0 } // ancre sur ce maillon (haut)
        ),
        prevBody, linkBody, true
    );
    joint.setContactsEnabled(false);

    prevBody = linkBody;
    prevAnchor = { x: 0, y: -0.25, z: 0 }; // bas du maillon
}

```

La chaîne montre le joint **le plus libre** : chaque maillon peut **tourner dans tous les axes** — c’est une rotule. Pas de moteur ici (le spherical n’en a pas en 3D), pas de limites : la chaîne tombe, oscille, s’enroule. Le bouton « Pousser » applique une impulsion latérale à tous les maillons — observez comment l’oscillation se propage le long de la chaîne, maillon par maillon. C’est le principe d’un **ragdoll** : un corps articulé sans contrôle moteur.

Station 4 — Le capteur raycast

Un rayon tournant qui scanne la scène

```

// Le rayon tourne autour de l'axe Y
let rayAngle = 0;

function updateRaycast() {
    rayAngle += raycastSpeed * FIXED_DT;

    // Direction dans le plan XZ, légèrement vers le bas
    const dir = {
        x: Math.cos(rayAngle),
        y: -0.3,
        z: Math.sin(rayAngle)
    };
    // Normaliser
    const len = Math.hypot(dir.x, dir.y, dir.z);
    dir.x /= len; dir.y /= len; dir.z /= len;

    // --- LE RAYCAST ---
    const ray = new RAPIER.Ray(origin, dir);
    const hit = world.castRay(ray, maxDist, true);

    if (hit) {
        const point = ray.pointAt(hit.timeOfImpact);
        // Dessiner le rayon jusqu'au point d'impact
        // Placer un marqueur rouge au point d'impact
    }
}

```

```
}  
}
```

Le capteur raycast est la seule station **sans joint** — c’est une **requête** du moteur physique, pas une contrainte. Un rayon vert tourne depuis un émetteur élevé et scanne les obstacles autour de lui. À chaque frame, `world.castRay()` retourne le premier collider touché, sa **distance** (`timeOfImpact`) et sa **normale**. Le marqueur rouge se place au point d’impact — c’est exactement le principe d’un **radar**, d’un **capteur de proximité**, ou d’un **lidar**. Aucune collision n’est générée : le raycast **lit** le monde, il ne le **modifie** pas.

💡 Le raycast en pratique

Dans un jeu, le raycast est partout : détection du sol sous le personnage (`castRay` vers le bas), visée FPS (vers l’avant), capteur de véhicule (vers l’avant), placement d’objet (depuis la souris). La station 4 montre le cas le plus **pur** : un rayon qui tourne et marque les impacts. En production, on ne dessine pas le rayon — on lit juste `hit.timeOfImpact` pour savoir « à quelle distance est l’obstacle ».

Partie 7 : Godot + Jolt — Joints & Moteurs

Les joints en Godot — des nœuds de scène

En Godot, un joint est un **nœud** (`Joint3D`) placé dans la scène, avec deux propriétés : `node_a` et `node_b` — les `NodePath` vers les deux bodies à relier. Le joint se positionne **automatiquement** entre les deux.

```
# Godot – HingeJoint3D (le revolute de Godot)
var hinge := HingeJoint3D.new()
hinge.node_a = base_body.get_path()
hinge.node_b = arm_body.get_path()
hinge.position = Vector3(0, 0.5, 0) # point d'attache

# Limites (en degrés, pas en radians !)
hinge.set_flag(HingeJoint3D.FLAG_USE_LIMIT, true)
hinge.set_param(HingeJoint3D.PARAM_LIMIT_LOWER, -80.0) # -80°
hinge.set_param(HingeJoint3D.PARAM_LIMIT_UPPER, 45.0) # +45°

# Moteur
hinge.set_flag(HingeJoint3D.FLAG_ENABLE_MOTOR, true)
hinge.set_param(HingeJoint3D.PARAM_MOTOR_TARGET_VELOCITY, 5.0) # rad/s
add_child(hinge)
```

Godot propose **cinq** types de joints 3D, tous héritant de `Joint3D` :

- **PinJoint3D** : point unique, rotation libre ($\approx$ spherical).
- **HingeJoint3D** : charnière ($\approx$ revolute).
- **SliderJoint3D** : translation le long d’un axe ($\approx$ prismatic).
- **ConeTwistJoint3D** : rotule avec limites de torsion (spherical + limites).
- **Generic6DOFJoint3D** : 6 degrés de liberté, chacun **lockable** ou **limitable**.

💡 Le Generic6DOFJoint3D — le couteau suisse

Le Generic6DOFJoint3D permet de **verrouiller ou limiter chaque axe** individuellement (3 translations + 3 rotations). Un revolute = on lock 5 axes, on en libère 1. Un prismatic = on lock 5, on libère 1 translation. Un spherical = on lock 3 translations, on libère 3 rotations. C'est l'outil **universel** — mais plus complexe à configurer. En pratique, on utilise les joints **spécialisés** quand ils existent, et le 6DOF pour les cas custom.

Moteurs en Godot — velocity vs position

Godot : moteur en vitesse uniquement

Contrairement à Rapier qui propose `configureMotorPosition` **et** `configureMotorVelocity`, Godot's HingeJoint3D n'a qu'un moteur en **vitesse** :

```
hinge.set_flag(HingeJoint3D.FLAG_ENABLE_MOTOR, true)
hinge.set_param(HingeJoint3D.PARAM_MOTOR_MAX_TORQUE, 100.0) # couple max
hinge.set_param(HingeJoint3D.PARAM_MOTOR_TARGET_VELOCITY, 5.0) # rad/s
```

Pour un **moteur en position** (servo), on utilise le Generic6DOFJoint3D avec `PARAM_ANGULAR_MOTOR_TARGET_VELOCITY` et un **PID maison** dans `_physics_process` : on ajuste la vitesse cible proportionnellement à l'écart angulaire. C'est plus de travail, mais c'est la flexibilité de Godot — on **code** l'asservissement au lieu de le configurer.

Partie 8 : Tableau comparatif des joints

Concept	Rapier (JS)	Godot 4 (GDScript)	Unity 6 (C#, PhysX)
Joint fixed	JointData.fixed(a1, f1, a2, f2)	FixedJoint3D (via 6DOF lock all)	FixedJoint
Joint revolute	JointData.revolute(a1, a2, axis)	HingeJoint3D	HingeJoint
Joint prismatic	JointData.prismatic(a1, a2, axis)	SliderJoint3D	SliderJoint
Joint spherical	JointData.spherical(a1, a2)	PinJoint3D / ConeTwistJoint3D	CharacterJoint
Joint 6DOF	— (composer plusieurs joints)	Generic6DOFJoint3D	ConfigurableJoint
Créer le joint	world.createImpulseJoint(desc, A, B)	add_child(joint); joint.node_a/b = ...	joint.connectedBody = rb
Ancres	Locales au body (Vector3)	joint.position (monde) + node_a/b	joint.anchor / connectedAnchor
Moteur vitesse	joint.configureMotorVelocity(factor)	PARAM_MOTOR_TARGET_VELOCITY + flag	joint.useMotor motor.targetVelocity
Moteur position	joint.configureMotorPosition(k, c)	(pos, PID maison dans _physics_process	joint.springDamper (Configurable)
Limites	joint.setLimits(min, max)	PARAM_LIMIT_LOWER / UPPER + flag	joint.limits.min / max
Supprimer	world.removeImpulseJoint(joint)	joint.queue_free()	Destroy(joint)
Raycast	world.castRay(ray, max, solid)	PhysicsRayQueryParameters3D + space.intersect_ray	Physics.Raycast

Table 1: Mêmes articulations, trois API. Rapier sépare **descripteur** et **création** ; Godot attache le joint comme **nœud de scène** ; Unity expose tout via le composant Joint. Les concepts sont identiques — seules les signatures diffèrent.

Partie 9 : Raycasts en Godot

PhysicsRayQueryParameters3D + intersect_ray

```
func _physics_process(_delta: float) -> void:
    var space := get_world_3d().direct_space_state
    var query := PhysicsRayQueryParameters3D.create(
        Vector3(0, 10, 0), # from
        Vector3(0, -10, 0), # to (POINT, pas direction !)
        0b0111             # collision mask
    )
    var hit := space.intersect_ray(query)
    if hit:
        var point := hit.position      # Vector3
        var normal := hit.normal      # Vector3
```

```
var collider := hit.collider    # le CollisionObject3D
print("Touché à ", point)
```

Godot prend **deux points** (from, to) au lieu d'origine + direction + distance. Le `collision_mask` (bitmask) filtre quels colliders sont testés — c'est l'équivalent des **collision groups** de Rapier. Le résultat est un **dictionnaire** avec position, normal, collider, rid (le RID du collider).

💡 Le collision mask — filtrer les raycasts

En Godot, chaque body a une `collision_layer` et un `collision_mask`. Le raycast a aussi un mask : il ne touche que les bodies dont la layer **correspond** au mask du raycast. C'est essentiel pour ignorer le personnage lui-même (le raycast de détection du sol ne doit pas toucher le joueur). En Rapier, on filtre avec les **collision groups** (`setQueryFilterGroups`).

Synthèse

💬 Ce qu'il faut retenir

- **Un joint** est une **contrainte** qui relie deux rigid bodies et supprime des degrés de liberté. Le solveur la résout à chaque `step()` au même titre que les contacts.
- **Quatre types** : fixed (soudure), revolute (charnière — 1 rotation), prismatic (tiroir — 1 translation), spherical (rotule — 3 rotations). Choisir selon le **mouvement** qu'on veut autoriser.
- **Les moteurs** animent un joint activement : `configureMotorVelocity` (régulateur) ou `configureMotorPosition` (servo). Les paramètres `stiffness` et `damping` sont le **ressort de la Session 7** — mais résolu comme contrainte, donc stable.
- **Les limites** bornent la course d'un joint (`setLimits(min, max)`) — les butées mécaniques. Sans elles, un joint peut tourner indéfiniment.
- **Les raycasts** interrogent le monde sans le modifier : `castRay` retourne le premier collider touché, sa distance et sa normale. Outil de détection du sol, de visée, de capteurs.
- **Rapier vs Godot** : Rapier sépare descripteur (`JointData`) et création (`createImpulseJoint`) ; Godot attache le joint comme nœud (`Joint3D + node_a/b`). Rapier a moteur en position **et** vitesse ; Godot n'a que la vitesse (le servo se code à la main).

💡 La suite

Session 16 : **contraintes avancées** et **IK** (cinématique inverse). On réutilisera les joints d'aujourd'hui pour construire des **chaînes articulées** (bras robot, ragdoll) et résoudre la question : « quels angles faut-il donner aux articulations pour que le pied atteigne ce point ? » Puis le TP véhicules (S17) — où les joints de roue (revolute + moteur) feront avancer une voiture.